STAGE 2 OF 4 · TAKE-HOME CASE STUDY

Build a small app. Show us how you think.

A self-contained app to bring to the next call to showcase your skills. The point isn't a polished submission but rather it's having real code that you know inside out, so we can sit down at stage 3 and extend it together.

TIME-BOX	SUBMIT WITHIN	STACK	RUNS VIA
2–3 hours	**~1** week	**PHP + Symfony** preferred	**Docker** mandatory

Goal

Build a CLI program that ingests a nested product feed, flattens it into a tabular shape, and writes it to a destination of your choice.

This is your chance to showcase how you approach a problem. Pick the patterns that make the case for you. DDD, CQRS, event-driven, hexagonal layering, whatever you'd actually reach for. Over-engineering is encouraged here. We want to see how you think and where you choose to invest, not the smallest possible solution.

Input data

A JSONL input file is attached to this brief. The structure has nested fields and your program is responsible for flattening it into row-shaped records.

What you build

A command-line program that:

1. Reads the input file (path passed as an argument or ENV variable)
2. Flattens the nested structure into rows
3. Writes the result to a destination
4. Logs errors

Output destinations

Pick whichever you find most interesting to implement. We don't want you fighting authentication setup.

- SQLite — write all data flat into a single table
- **CSV** or **XML** file
- Google Sheets — fine if you really want, but skip it if it means you'll spend time on OAuth. Writing to a public sheet is fine.

Constraints

- Docker is mandatory. We must be able to run your solution with docker compose up on a fresh machine with no host dependencies beyond Docker itself.
- PHP + Symfony are preferred, but not required. What matters is that your chosen stack lets you showcase your engineering.
- Time-box: aim for 2–3 hours. If you want to go deeper, that's fine but past 3 hours we'd rather you write a short note on what you'd do next than keep grinding.

What we look for

- Clean code with deliberate architectural choices
- Tests where they meaningfully demonstrate something (not coverage for coverage's sake)
- A short README:
  - How to run it
  - Why you made the design choices you did
  - What you'd do with more time
  - Any AI assistance disclosure (see below)

AI usage

AI assistants are welcome but disclose what you used and where. We care about how you direct the AI, not whether you did. Hiding AI usage is a stronger negative signal than using it.

The live interview

In the follow-up interview, **you'll be asked to extend your code live**, paired with the hiring manager. Plan accordingly:

- Know your code well enough to extend it confidently
- Be ready to explain why you made each choice and to defend or revise it
- Anything you can't explain shouldn't be in the codebase

The code is the conversation starter; the conversation is the signal.

Submission

Send a GitHub / GitLab link, git bundle or zip including:

- Source code
- Dockerfile + compose file if needed
- README (with AI usage notes)